

Lab 3: Architectural Justification

Student Club Event Ticketing & Budget Portal | Sathvik Karthik Malali | PES1UG24CS616 | Sem 5, Sec K

Architecture Selection: "We chose Microservices Architecture for the Student Club Event Ticketing & Budget Portal System."

Architectural Choice

The system is decomposed into independently deployable microservices: Proposal & Approval Service, Budget Service, Ticketing Service, Check-In Service, and an Auth & RBAC Service. Each has its own data store, fronted by the Club Lead Portal and Campus Admin Scanner client applications.

Two Reasons for This Choice

- **Independent scaling for divergent load profiles:** The Check-In Service must validate QR tickets in under 100ms during bursts of scans at event start (NFR001), while the Proposal & Approval Service sees low, sporadic traffic tied to the three-stage sign-off workflow (Faculty Coordinator → Finance Office → Dean). Microservices let us scale Check-In horizontally for peak attendance without over-provisioning the approval workflow.
- **Fault isolation across unrelated concerns:** Ticket check-in at the venue must keep working even if the budget-approval workflow is degraded (e.g., a Finance Office outage), and vice versa. Since each microservice fails independently, a fault in one subsystem cannot block event entry or the approval audit trail.

Security Advantage

Every service independently validates a role-scoped token issued by the Auth & RBAC Service before serving a request, enforcing NFR002 (a Club Lead may only view/manage their own club's proposals, budgets, and tickets) at each service boundary rather than in one shared codebase. Because each service also owns its own database, a compromised or buggy service (e.g., Ticketing) cannot directly query another service's data (e.g., Budget), which contains the blast radius of any single breach.

Performance Benefit

The Check-In Service can be deployed, cached, and scaled independently (e.g., an in-memory index of active ticket IDs) to meet the sub-100ms validation target of NFR001, without being slowed down by heavier, transactional workloads such as multi-stage approval routing or itemized budget calculations running in other services.